

A Field Guide to the MDD Pipeline

What every piece of the code does, why it is there, and the mathematics underneath

REST-meta-MDD SURGE Project • June 8, 2026

Abstract

A companion document to the REST-meta-MDD classification and severity-regression pipeline. Written to be read start to finish: each section explains the intuition first, then the formal machinery, and ties every concept to the function in the code that implements it.

Contents

1. The problem, and the one idea that shapes everything	1
2. From brain scans to numbers: feature construction	1
3. The villain of the story: site effects and data leakage	4
4. Leakage-free preprocessing	4
5. Honest validation: leave-one-site-out	5
6. The models	6
7. Multimodal fusion	8
8. Measuring success: the metrics	8
9. Opening the black box: interpretability	9
10. What good results actually look like	10
11. References	10

1. The problem, and the one idea that shapes everything

We are trying to do two things with resting-state brain imaging. First, **classification**: given a person's MRI-derived features, decide whether they have Major Depressive Disorder (MDD) or are a healthy control. Second, **regression**: among people who do have MDD, predict how severe it is, measured by their Hamilton Depression Rating Scale (HAMD) total score.

There is one idea that governs every design decision in this codebase, and it is worth stating before any mathematics: **the data come from 25 different scanners, and the scanner leaves a bigger fingerprint on the data than depression does**. Each site has its own MRI machine, its own field strength, its own sequence parameters, its own ambient noise. If you are not careful, a machine-learning model will learn to recognise *which scanner* a scan came from, and because different sites happened to recruit different proportions of patients, recognising the scanner is almost as good as guessing the diagnosis — without the model ever learning anything about the brain.

Everything that follows — the harmonization, the leave-one-site-out validation, the insistence on fitting transforms inside cross-validation folds — exists to stop the model from cheating in that way. The pipeline is built to be *honest* rather than to produce an impressive number. A correct model here scores in the high sixties to low seventies for balanced accuracy; a model reporting ninety-plus percent has almost certainly been fooled by the scanner.

2. From brain scans to numbers: feature construction

Machine-learning models eat vectors of numbers, not brain images. So the first job is to turn each subject's imaging into features. We build two *modalities* — functional and structural — because they capture different things, and a person can show abnormalities in one but not the other.

2.1 Functional connectivity from ROI time series

The richest functional feature is **functional connectivity (FC)**: a measure of how synchronously two brain regions fluctuate over time. The raw input is a matrix of shape 200×1833 per subject — 200 time points, and 1833 region time series stacked from several brain atlases. We slice out the 116 columns belonging to the **AAL atlas** (Tzourio-Mazoyer et al., 2002), giving a clean 200×116 matrix: 200 time points for each of 116 regions.

For each pair of regions i and j , we compute the **Pearson correlation** of their time series $x_i(t)$ and $x_j(t)$:

$$r_{ij} = \frac{\sum_{t=1}^T (x_i(t) - \bar{x}_i)(x_j(t) - \bar{x}_j)}{\sqrt{\sum_t (x_i(t) - \bar{x}_i)^2} \sqrt{\sum_t (x_j(t) - \bar{x}_j)^2}}$$

This number lives in $[-1, 1]$: $+1$ means the two regions rise and fall in perfect lockstep, -1 means they are perfectly anti-correlated, 0 means unrelated. Collecting all pairs gives a 116×116 symmetric **connectivity matrix**.

Why we then apply the Fisher z-transform. A correlation coefficient is a badly behaved quantity for statistics: its sampling variance depends on its own value (correlations near ± 1 are squeezed against the boundary and have tiny variance, correlations near 0 have large variance). Averaging or modelling such values is treacherous. Fisher’s transformation (Fisher, 1915) fixes this:

$$z = \operatorname{arctanh}(r) = \frac{1}{2} \ln \frac{1+r}{1-r}$$

After this transform the values are approximately normally distributed with a variance that no longer depends on the underlying correlation, $\operatorname{Var}(z) \approx \frac{1}{T-3}$. This *variance stabilization* is why nearly every connectivity study works in z -space rather than r -space, and why the code does `np.arctanh(...)`.

Why we keep only the upper triangle. The connectivity matrix is symmetric ($r_{ij} = r_{ji}$) and its diagonal is trivially 1. All the information lives in the strict upper triangle, so we vectorise just those entries:

$$\# \text{features} = \binom{116}{2} = \frac{116 \times 115}{2} = 6670.$$

So each subject becomes a 6,670-dimensional functional-connectivity vector. (The code’s `fc_upper` does exactly this with `np.triu_indices(n, k=1)`.)

2.2 Voxelwise functional metrics, reduced to regions

Beyond connectivity *between* regions, the dataset provides five voxelwise maps describing local activity. Each is a $61 \times 73 \times 61$ image. Briefly, with their mathematics:

- **ALFF** — Amplitude of Low-Frequency Fluctuations (Zang et al., 2007). The strength of slow spontaneous activity, computed as the averaged amplitude of the signal’s power spectrum within the low-frequency band $[0.01, 0.08]$ Hz:

$$\text{ALFF} = \frac{1}{|\mathcal{F}|} \sum_{f \in [0.01, 0.08]} |X(f)|,$$

where $X(f)$ is the Fourier transform of the voxel time series.

- **fALFF** — Fractional ALFF (Zou et al., 2008). The same low-frequency amplitude but expressed as a *fraction* of the total spectral amplitude, which suppresses physiological noise:

$$\text{fALFF} = \frac{\sum_{f \in [0.01, 0.08]} |X(f)|}{\sum_{f \in [0, f_{\text{Nyq}}]} |X(f)|}.$$

- **ReHo** — Regional Homogeneity (Zang et al., 2004). How similar a voxel’s time series is to its immediate neighbours, measured by **Kendall’s coefficient of concordance** over a neighbourhood of K voxels across n time points:

$$W = \frac{\sum_{t=1}^n (R_t - \bar{R})^2}{\frac{1}{12} K^2 (n^3 - n)},$$

where R_t is the sum of ranks of the K voxels at time t and $\bar{R}$ their mean. $W \in [0, 1]$; high values mean locally synchronised activity.

- **VMHC** — Voxel-Mirrored Homotopic Connectivity (Zuo et al., 2010). The correlation between a voxel and its mirror-image voxel in the opposite hemisphere, Fisher-z transformed. It measures how tightly the two hemispheres cooperate.
- **Degree Centrality (DC)** — A graph measure of how much of a “hub” a voxel is (Buckner et al., 2009): the number (or summed strength) of connections it has above a threshold τ ,

$$DC_i = \sum_{j \neq i} \mathbb{1}[r_{ij} > \tau].$$

These are voxelwise, but classical models cannot digest hundreds of thousands of voxels without overfitting. So we **parcellate**: for each AAL region R_r , we average the metric over the valid brain voxels it contains,

$$m_r = \frac{1}{|R_r|} \sum_{v \in R_r} \text{metric}(v),$$

turning each $61 \times 73 \times 61$ map into 116 numbers. Five metrics $\times$ 116 regions gives **580 features** (the `roi_metrics` block; the code’s `roi_means`). This is the right granularity: regional, interpretable, and matched to the connectivity atlas.

2.3 Structural gray matter from VBM

Depression may also leave structural traces — subtle differences in gray-matter volume. **Voxel-Based Morphometry (VBM)** quantifies this. The dataset provides `mwc1` files: *modulated, warped, class-1 (gray matter)* maps. The word **modulated** matters mathematically. When a brain is warped into a common template, regions are stretched and squeezed; modulation multiplies each voxel’s tissue probability by the **Jacobian determinant** $|J(v)|$ of the deformation field to *preserve the original local volume*:

$$\text{mwc1}(v) = \text{wc1}(v) \cdot |J(v)|.$$

Without modulation you would be comparing tissue *density* in template space; with it you compare true *volume*, which is what morphometry is about. This is why the code uses `mwc1` specifically and not the native-space `c1` or the un-modulated `wc1`. We parcellate these the same way, giving **116 structural features** per subject.

2.4 The atlas, and why interpolation type is not a detail

To parcellate any voxelwise map we overlay the AAL atlas — an image whose voxel values are integer *labels* $(1, 2, \dots, 116)$ marking which region each voxel belongs to. The atlas usually ships at a finer resolution than our $61 \times 73 \times 61$ data, so it must be resampled to match.

Here is a trap worth understanding. If you resample a *label* image with linear interpolation, a voxel sitting between region 45 and region 46 becomes 45.5 — a label that means nothing. The fix is **nearest-neighbour interpolation**, which simply takes the label of the closest source voxel and never invents fractional labels. The code’s `prepare_atlas` enforces `interpolation="nearest"` for this reason. (Continuous metric maps, by contrast, are fine with linear interpolation — and the code uses linear for those.)

3. The villain of the story: site effects and data leakage

3.1 What a site effect is, mathematically

Imagine the value of feature g for subject j at site i decomposes as

$$y_{ijg} = \underbrace{\alpha_g}_{\text{true biology}} + \underbrace{\gamma_{ig}}_{\text{additive site offset}} + \underbrace{\delta_{ig} \varepsilon_{ijg}}_{\text{site-scaled noise}}.$$

The terms γ_{ig} (each scanner shifts the feature by some amount) and δ_{ig} (each scanner inflates or deflates its variance) are pure nuisance. They have nothing to do with depression. But they are *large* — often larger than the biological signal — and they are *systematic*, so a model can latch onto them.

3.2 Why that becomes leakage

Data leakage is when information from the test set sneaks into training, making the model look better than it truly is. The most insidious form here is subtle: if you estimate any statistic — a feature mean, a scaling factor, a harmonization parameter — using the *whole* dataset before splitting into train and test, then the test subjects have influenced the transformation applied to themselves. The model has, in effect, peeked.

Mathematically, suppose you standardise a feature using the global mean μ_{all} computed over every subject. The test residual $x_{\text{test}} - \mu_{\text{all}}$ depends on μ_{all} , which depends on the test data. Train and test are no longer independent, and the cross-validation estimate of performance is biased upward. The cure is simple to state and easy to get wrong: **every quantity must be estimated on the training fold alone**. The entire `FoldTransform` class exists to guarantee this.

4. Leakage-free preprocessing

The pipeline applies four transformations, in order, and each is fit on the training fold and merely *applied* to the test fold.

4.1 Confound regression

Age, sex, education, and head motion all differ between patients and controls and all affect brain features. We do not want the model to “detect depression” by secretly detecting that patients are older or move more. So we **regress these confounds out** of every feature using ordinary least squares.

Stack the confounds (plus an intercept) into a design matrix C . For the feature matrix X , the least-squares coefficients are

$$\hat{\beta} = (C^\top C)^{-1} C^\top X,$$

and the residualised features are

$$\tilde{X} = X - C\hat{\beta} = (I - C(C^\top C)^{-1} C^\top)X = (I - H)X,$$

where $H = C(C^\top C)^{-1} C^\top$ is the **hat matrix**, the orthogonal projector onto the column space of the confounds. Subtracting HX projects the data onto the subspace *orthogonal* to the confounds — geometrically, we remove exactly the component of each feature that the confounds can explain. Crucially, $\hat{\beta}$ is learned on training subjects and the *same* $\hat{\beta}$ is applied to test subjects (the code’s `ConfoundRegressor.fit / .transform`), so no test information leaks.

4.2 Site harmonization (ComBat)

Confound regression handles continuous nuisances, but the scanner-specific γ_{ig} and δ_{ig} need dedicated treatment. The standard tool is **ComBat** (Johnson, Li & Rabinovic, 2007), borrowed from genomics and

brought to neuroimaging by Fortin et al. (2017, 2018). ComBat models exactly the decomposition from §3.1 and removes the site terms while protecting biological covariates:

$$y_{ijg}^* = \frac{y_{ijg} - \hat{\alpha}_g - C_{ij}\hat{\beta}_g - \hat{\gamma}_{ig}^*}{\hat{\delta}_{ig}^*} + \hat{\alpha}_g + C_{ij}\hat{\beta}_g.$$

The starred parameters $\hat{\gamma}^*, \hat{\delta}^*$ are **empirical-Bayes** estimates: rather than trusting each site’s raw mean and variance (noisy when a site has few subjects), ComBat shrinks them toward a prior pooled across all features, which stabilises the estimates. This shrinkage is ComBat’s signature contribution.

The implementation here (LSHarmonizer) provides the transparent **location/scale core** of this model — it removes the additive and multiplicative site effects and protects covariates — but omits the empirical-Bayes shrinkage for clarity and testability; the documentation points to `neuroHarmonize` for the full version in production.

The honest subtlety under leave-one-site-out. You cannot legitimately harmonize a site you have never seen — there is no $\hat{\gamma}, \hat{\delta}$ for it. The code therefore leaves an unseen test site *unharmonized by default* and warns you, rather than silently fabricating parameters. (An optional transductive mode estimates them from the test site’s own features, using no labels; if you use it, you disclose it.)

4.3 Standardization

Models like the SVM are sensitive to feature scale. We z-score each feature using the training mean and standard deviation:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

Again, μ_{train} and σ_{train} come from training data only (`StandardScalerSafe`), with a guard so that near-constant features (which would divide by zero) are left alone.

4.4 Optional univariate feature selection

With 6,670 functional features and only ~1,600 subjects, one may wish to keep the most informative features. The code’s `univariate_topk_idx` ranks features by the absolute **point-biserial correlation** between each feature and the binary label — mathematically identical to Pearson correlation with a 0/1 target — and keeps the top k . This selection is performed *inside* the fold on training data only; selecting features using the full dataset is a classic and severe leak.

5. Honest validation: leave-one-site-out

5.1 What cross-validation estimates

Cross-validation repeatedly splits the data into a training part and a held-out test part, trains on the former, evaluates on the latter, and averages. It estimates how well the model will do on *data it has not seen*.

5.2 Why the splits must respect sites

The question we actually care about clinically is: *will this model work at a new hospital with a new scanner?* The validation scheme should mimic that. **Leave-one-site-out (LOSO)** holds out one entire site as the test set and trains on the other 24, rotating through all sites:

$$\widehat{\text{err}}_{\text{LOSO}} = \frac{1}{S} \sum_{s=1}^S \text{err}(\text{model trained on sites } \neq s, \text{ tested on site } s).$$

Because the held-out site never appears in training, the model cannot exploit its scanner fingerprint. This is the honest estimate, implemented via `scikit-learn`’s `LeaveOneGroupOut` grouped by site.

5.3 k-fold as a deliberate upper bound

The pipeline also runs ordinary stratified **k-fold**, which shuffles subjects across sites so that the same site appears in both train and test. This *allows* the scanner-fingerprint leak — and that is the point. The gap

$$\Delta = \widehat{\text{acc}}_{\text{kfold}} - \widehat{\text{acc}}_{\text{LOSO}}$$

quantifies how much of the apparent performance is site leakage rather than biology. In our validation experiment on synthetic data with a built-in site confound, k-fold reported 0.68 while LOSO sat at chance (0.50) — a 0.18 gap that is itself a publishable finding about why the inflated numbers in the literature should be distrusted.

5.4 Nested cross-validation for fusion

When a second model (the fusion meta-learner, §7) is trained on the *outputs* of the base models, those outputs must themselves be out-of-sample, or the meta-learner sees leaked predictions. So inside each training fold we run an **inner** cross-validation to generate out-of-fold base predictions, fit the meta-learner on those, and only then apply it to the untouched outer test fold. This nesting (`_inner_oof_scores`) is the part practitioners most often get wrong.

6. The models

All models share one interface: `fit(X, y)` then `score(X)`, where `score` returns $P(\text{MDD})$ for classification or the predicted value for regression. This uniformity lets the cross-validation engine treat them identically.

6.1 Linear Support Vector Machine — the honest baseline

The linear SVM (Cortes & Vapnik, 1995) finds the hyperplane $w^\top x + b = 0$ that separates the two classes with the widest possible margin. Formally it solves

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i (w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0,$$

equivalently, in unconstrained **hinge-loss** form,

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i (w^\top x_i + b)).$$

The term $\frac{1}{2} \|w\|^2$ maximises the margin (a form of regularisation), and C trades off margin width against training errors. We use the **linear** kernel deliberately: with thousands of features and few subjects, a linear boundary is already extremely flexible, it resists overfitting, and — critically — its weight vector can be interpreted (after the Haufe correction in §9). This is the model every fancier method must beat.

6.2 Logistic regression

Logistic regression models the probability of MDD directly through the **sigmoid**:

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}.$$

It is fit by minimising the **cross-entropy (log-loss)** with L_2 regularisation:

$$\min_w \sum_i -[y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|^2, \quad p_i = \sigma(w^\top x_i + b).$$

It is fast, naturally probabilistic, and its coefficients give a transparent (if, again, correlation-confounded) story.

6.3 Ridge regression — for HAMD severity

For the continuous HAMD target we use **ridge regression**: ordinary least squares with an L_2 penalty that shrinks coefficients and tames the high-dimensional, correlated feature space:

$$\min_w \|y - Xw\|^2 + \lambda \|w\|^2, \quad \hat{w} = (X^\top X + \lambda I)^{-1} X^\top y.$$

The λI term makes $X^\top X + \lambda I$ invertible even when features outnumber subjects, which plain least squares cannot handle.

6.4 XGBoost — the nonlinear contender

XGBoost (Chen & Guestrin, 2016) builds an **additive ensemble of decision trees**, each new tree correcting the errors of the ones before:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F} \text{ (regression trees)}.$$

It minimises a **regularised objective** using a second-order Taylor expansion of the loss at each boosting step:

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2,$$

where T is the number of leaves and the penalties discourage overly complex trees. It captures nonlinear interactions a linear model cannot — but at this sample size it rarely beats the SVM under honest LOSO, which is exactly the comparison we want to make explicit.

6.5 The Graph Neural Network — brain as a network

The most “state-of-the-art” model treats each subject’s brain as a **graph**: nodes are the 116 AAL regions, node features are that region’s metric values (ALFF, fALFF, ReHo, VMHC, DC), and edges connect strongly-connected regions (the top- k strongest $|FC|$ values per node).

A **Graph Convolutional Network** (Kipf & Welling, 2017) updates each node by mixing its own features with its neighbours’ features, layer by layer:

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}),$$

where $\tilde{A} = A + I$ adds self-loops, $\tilde{D}$ is its diagonal degree matrix, $H^{(l)}$ holds the node features at layer l , $W^{(l)}$ is a learned weight matrix, and σ is a nonlinearity (ReLU). The normalisation $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ keeps feature magnitudes stable as information propagates.

The **Graph Attention Network** (Veličković et al., 2018) goes further, *learning* how much each neighbour matters rather than weighting them all equally:

$$\alpha_{ij} = \text{softmax}_j \left(\text{LeakyReLU} \left(a^\top [Wh_i \parallel Wh_j] \right) \right), \quad h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \right),$$

where $\parallel$ is concatenation and a is a learned attention vector. To turn the per-node representations into one prediction per subject, we apply **global mean pooling**:

$$h_G = \frac{1}{|V|} \sum_{v \in V} h_v,$$

followed by a small fully-connected head. This design echoes BrainGNN (Li et al., 2021). It is included for completeness and comparison — with the caveat, stated honestly throughout, that more parameters than

subjects is a recipe for overfitting, and demonstrating that rigorously is a legitimate result rather than a failure.

7. Multimodal fusion

We have two views of each brain — functional and structural — and want to combine them. There are three places one can fuse:

- **Early fusion** concatenates the raw feature vectors. Simple, but the 7,000-dimensional functional block drowns the 116-dimensional structural block.
- **Intermediate / joint fusion** learns a shared neural representation. Powerful, but it overfits badly at our sample size.
- **Late fusion** trains a separate model per modality and combines their *predictions*. This is the robust choice here.

The pipeline uses late fusion via **stacking**. Each modality's base model produces a score; a small meta-learner g (logistic regression for classification, ridge for regression) learns how to weight them:

$$\hat{p}_{\text{fused}} = g(s_{\text{functional}}, s_{\text{structural}}).$$

The meta-learner is trained on the *out-of-fold* base scores from the inner cross-validation (§5.4), never on scores it will be tested on. A weak modality cannot drag down a strong one, because the meta-learner simply learns a small weight for it — which is precisely the behaviour our synthetic-data test confirmed: when the structural modality carried a genuine signal and the functional one did not, the fused score tracked the structural model and did not hallucinate extra accuracy. Expect the realistic gain from fusion to be modest (one to three points), because structural differences in depression are small (ENIGMA-MDD effect sizes around $d \approx 0.1$ – 0.2 ; Schmaal et al., 2017).

8. Measuring success: the metrics

8.1 Classification

From the confusion-matrix counts — true/false positives and negatives (TP, FP, TN, FN) — we report:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy alone is misleading when classes are imbalanced (a site that is 80% patients lets a lazy “always MDD” model score 80%). So the headline number is **balanced accuracy**, the average of sensitivity and specificity:

$$\text{Balanced Acc} = \frac{1}{2} \left(\underbrace{\frac{TP}{TP + FN}}_{\text{sensitivity}} + \underbrace{\frac{TN}{TN + FP}}_{\text{specificity}} \right).$$

We also report **precision**, **recall**, and their harmonic mean **F1**:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Two threshold-free curves summarise performance across all decision thresholds. **ROC-AUC** is the area under the curve of true-positive rate versus false-positive rate; it equals the probability that a randomly chosen patient is scored higher than a randomly chosen control:

$$\text{ROC-AUC} = \Pr(s(\text{patient}) > s(\text{control})).$$

PR-AUC (average precision) is the area under the precision-recall curve and is more informative than ROC-AUC under class imbalance.

8.2 Regression

For HAMD severity we report the mean absolute error, root-mean-square error, coefficient of determination, and the correlation between predicted and true scores:

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_i |y_i - \hat{y}_i|, & \text{RMSE} &= \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \\ R^2 &= 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, & r &= \frac{\sum_i (\hat{y}_i - \bar{\hat{y}})(y_i - \bar{y})}{\sqrt{\sum_i (\hat{y}_i - \bar{\hat{y}})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}. \end{aligned}$$

RMSE punishes large errors more than MAE; R^2 tells you what fraction of the variance in severity your model explains (negative R^2 means you would do better predicting the mean — a real possibility for this genuinely hard target).

9. Opening the black box: interpretability

A clinically useful model should tell us *which brain features* drive its decisions, so they can be checked against neuroscience.

9.1 Why you cannot read a linear model's weights directly

It is tempting to say “the SVM gave feature 1024 a big weight, so that connection matters.” This is **wrong** when features are correlated — and connectivity features are heavily correlated. The reason, made precise by Haufe et al. (2014), is that a discriminative (backward) model assigns weights to *cancel noise*, not to *reflect signal*; a feature carrying pure noise can receive a large weight simply because it helps subtract noise from a genuinely informative feature.

9.2 The Haufe transformation

Haufe et al. showed how to convert the discriminative weight vector w into an interpretable **forward (activation) pattern** A that reflects how the underlying signal actually expresses itself in each feature. For a latent variable $\hat{s} = w^\top x$, the forward pattern is

$$A = \Sigma_x w \Sigma_{\hat{s}}^{-1},$$

where Σ_x is the covariance of the features and $\Sigma_{\hat{s}}$ the (scalar) variance of the latent. For a single output this reduces to

$$A \propto \Sigma_x w,$$

which is exactly what the code's `haufe_transform` computes — the data covariance times the weight vector, normalised. *Always* apply this before mapping weights back to brain regions; skipping it is a mistake reviewers will catch.

9.3 SHAP and permutation importance

For any model, including the tree ensembles and the GNN, we can use **SHAP values** (Lundberg & Lee, 2017), grounded in cooperative game theory. The Shapley value of feature i is its average marginal contribution across all possible feature coalitions:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)].$$

This is the unique attribution satisfying a set of fairness axioms (efficiency, symmetry, dummy, additivity). A cheaper, model-agnostic alternative provided in `evaluate.py` is **permutation importance**: shuffle one feature, measure how much performance drops, and repeat,

$$I_j = \mathcal{M}_{\text{baseline}} - \mathbb{E}[\mathcal{M}_{\text{feature } j \text{ permuted}}].$$

9.4 Back to the brain

Finally, `edge_index_to_roi_pair` maps an important connectivity feature (an index into the upper-triangle vector) back to the pair of AAL regions it represents, so findings can be read anatomically and checked against the depression literature — default-mode, salience, and limbic-prefrontal circuits. Convergence with known biology is the validity check; wildly implausible “important” regions usually signal that the model is explaining residual site noise rather than disease.

10. What good results actually look like

It is worth ending where we began. Under honest leave-one-site-out validation:

- MDD-vs-HC balanced accuracy of **65–73%** is a *good, credible* result. Numbers above the high seventies should trigger a hunt for leakage, not celebration.
 - HAMD regression will show a **modest positive correlation** and possibly a near-zero or negative R^2 , because predicting severity from imaging is genuinely hard and the target range is restricted to patients.
 - Multimodal fusion will typically add **one to three points** over the best single modality.
 - The most defensible contribution of this work is not a record accuracy but a *rigorous, leakage-free, multi-pipeline comparison* — functional versus structural, fused, and global-signal-regressed versus not — reported with the LOSO/k-fold gap laid bare. That honesty is the result.
-

11. References

Standard references for the methods used. Verify exact volume/page details against the original sources before formal submission.

- Buckner, R. L., et al. (2009). Cortical hubs revealed by intrinsic functional connectivity. *Journal of Neuroscience*, 29(6), 1860–1873. — degree centrality
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*. — XGBoost
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. — SVM
- Fisher, R. A. (1915). Frequency distribution of the values of the correlation coefficient in samples from an indefinitely large population. *Biometrika*, 10(4), 507–521. — Fisher z-transform
- Fortin, J.-P., et al. (2017/2018). Harmonization of multi-site diffusion tensor imaging / cortical thickness data. *NeuroImage*. — neuroComBat / neuroHarmonize
- Haufe, S., et al. (2014). On the interpretation of weight vectors of linear models in multivariate neuroimaging. *NeuroImage*, 87, 96–110. — Haufe transformation
- Jenkinson, M., et al. (2002). Improved optimization for the robust and accurate linear registration and motion correction of brain images. *NeuroImage*, 17(2), 825–841. — FD (Jenkinson)

- Johnson, W. E., Li, C., & Rabinovic, A. (2007). Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics*, 8(1), 118–127. — ComBat
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *ICLR*. — GCN
- Li, X., et al. (2021). BrainGNN: Interpretable brain graph neural network for fMRI analysis. *Medical Image Analysis*, 74. — brain GNN
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*. — SHAP
- Power, J. D., et al. (2012). Spurious but systematic correlations in functional connectivity MRI networks arise from subject motion. *NeuroImage*, 59(3), 2142–2154. — framewise displacement / motion
- Schmaal, L., et al. (2017). Cortical abnormalities in adults and adolescents with major depression. *Molecular Psychiatry*, 22, 900–909. — ENIGMA-MDD structural effect sizes
- Tzourio-Mazoyer, N., et al. (2002). Automated anatomical labeling of activations in SPM using a macroscopic anatomical parcellation of the MNI MRI single-subject brain. *NeuroImage*, 15(1), 273–289. — AAL atlas
- Varoquaux, G., et al. (2010). Detection of brain functional-connectivity difference in post-stroke patients using group-level covariance modeling. *MICCAI*. — tangent-space connectivity
- Veličković, P., et al. (2018). Graph attention networks. *ICLR*. — GAT
- Yan, C.-G., et al. (2019). Reduced default mode network functional connectivity in patients with recurrent major depressive disorder. *PNAS*, 116(18), 9078–9083. — REST-meta-MDD
- Zang, Y., et al. (2004). Regional homogeneity approach to fMRI data analysis. *NeuroImage*, 22(1), 394–400. — ReHo
- Zang, Y.-F., et al. (2007). Altered baseline brain activity in children with ADHD revealed by resting-state functional MRI. *Brain and Development*, 29(2), 83–91. — ALFF
- Zou, Q.-H., et al. (2008). An improved approach to detection of amplitude of low-frequency fluctuation (ALFF) for resting-state fMRI: fractional ALFF. *Journal of Neuroscience Methods*, 172(1), 137–141. — fALFF
- Zuo, X.-N., et al. (2010). Growing together and growing apart: regional and sex differences in the lifespan developmental trajectories of functional homotopy. *Journal of Neuroscience*, 30(45), 15034–15043. — VMHC